Securing Agent-Generated Code Through Development History

Prateek Rajput*, Shahzeb Khan†, Jacques Klein*, and Tegawendé F. Bissyandé*
*University of Luxembourg, Luxembourg
{prateek.rajput, jacques.klein, tegawende.bissyande}@uni.lu
†Nostrum Biodiscovery S.L., Barcelona, Spain
shahzeb.khan@nostrumbiodiscovery.com

***Abstract*—Agentic software development makes it possible to capture the process of code creation, moving beyond static comments and commit messages to preserve edits, tool interactions, and recorded ReAct-style reasoning behind implementation decisions. We present TRACER (Trace-based Runtime Analysis for Continuous Evolution and Release), a provenance-aware workflow that links task descriptions and recorded agent rationale to the evolution of code and its constituent symbols. We ask whether combining these reasoning traces with code evolution helps investigators understand implementation assumptions and better diagnose and fix security flaws. TRACER preserves diagnostic evidence, repair rationale, and associated executable boundary checks alongside each repair, providing persistent context for future updates, refactors, and feature additions. Across 3,096 repair runs on the SusVibes benchmark, TRACER's change-informed planning covers reference-vulnerable symbols in 94.6% of attempts, compared with 13.4% for stock SAST locations. The rationale-enabled configuration achieves the highest verified security pass rates among the evaluated configurations with both investigators, while a stage-level analysis exposes where remaining candidates are lost during diagnosis and validation. Extending TRACER to real-world structural-biology repositories, we verify five component repairs and evaluate subsequent feature evolution under silent-fix conditions, where pairing persistent repair memory with a boundary gate and one feedback retry improves joint feature-and-security-check success from 3/5 to 5/5.**

***Index Terms*—Agentic Software Engineering, Vulnerability Repair, Provenance and Reasoning Traces, Silent Security Fixes, Continuous Software Evolution, Static and Dynamic Analysis.**

I. INTRODUCTION

Autonomous coding agents can navigate repositories, edit source files, and execute test suites through interfaces such as SWE-agent [1], while agent-based repair systems extend this tool-driven workflow to debugging [2]. For engineering teams adopting these systems, security remains a critical review obligation. Empirical studies of generated code and coding-assistant use document security weaknesses [3]–[5]. A patch can satisfy functional tests while leaving an unsafe input boundary, incomplete validation, or a violated API contract. SusVibes makes this gap explicit by evaluating functionality and security separately on repository-level tasks [6], [7]. Teams accepting working, agent-generated changes therefore need to uncover and repair omitted security conditions without breaking the requested functionality.

Existing software engineering defenses provide several starting points. Static application security testing (SAST) tools such as CodeQL [8] and Semgrep [9] supply reproducible rules, source locations, and flow evidence. Industrial deployments and developer studies emphasize integrating actionable diagnostics into code review [10]–[12]. LLM-assisted taint and dataflow analysis [13]–[15], analyzer-guided repair [16], and program-analysis-guided proof-of-concept generation [17] further connect model judgments to source and execution evidence. These techniques help examine the implementation under review, but current-source findings alone do not explain why a coding agent omitted a guard or what it assumed about upstream inputs. When scanner locations also miss the relevant changed symbols, an investigator must first establish where to look and then reconstruct the missing security condition.

Agentic development offers the recorded process of code creation as an additional source of evidence. A trajectory can preserve intermediate edits, tool interactions, and interleaved ReAct-style explanations [18], extending the evidence available from a final diff and commit message. Fine-grained AST differencing makes structural changes explicit [19], while recorded rationale supplies statements about input trust, callers, and library behavior. Together, these channels can expose a removed guard, an asymmetry between sibling functions, or a mismatch between an agent's stated assumption and its implementation. We treat explanations as hypotheses to check against current source, since generated rationale need not faithfully reveal internal reasoning [20], [21]. The opportunity is to connect the history of a change to the present security contract that a repair must satisfy.

Repair is also a maintenance problem. Work on silent vulnerability fixes shows that security-relevant changes can lack an explicit security designation [22], and subsequent edits can undo an earlier repair, as explored in security-regression studies [23]. A later refactor or feature addition may preserve the requested behavior while losing a condition whose purpose is no longer apparent. The same provenance collected during investigation can carry that purpose forward. The diagnostic evidence, enforced invariant, and independently owned executable boundary check can remain linked to the repaired symbol. A maintenance workflow can then test subsequent changes against the saved contract and return concrete feedback when they violate it.

We present TRACER (*Trace-based Runtime Analysis for Continuous Evolution and Release*), a provenance-aware secu-

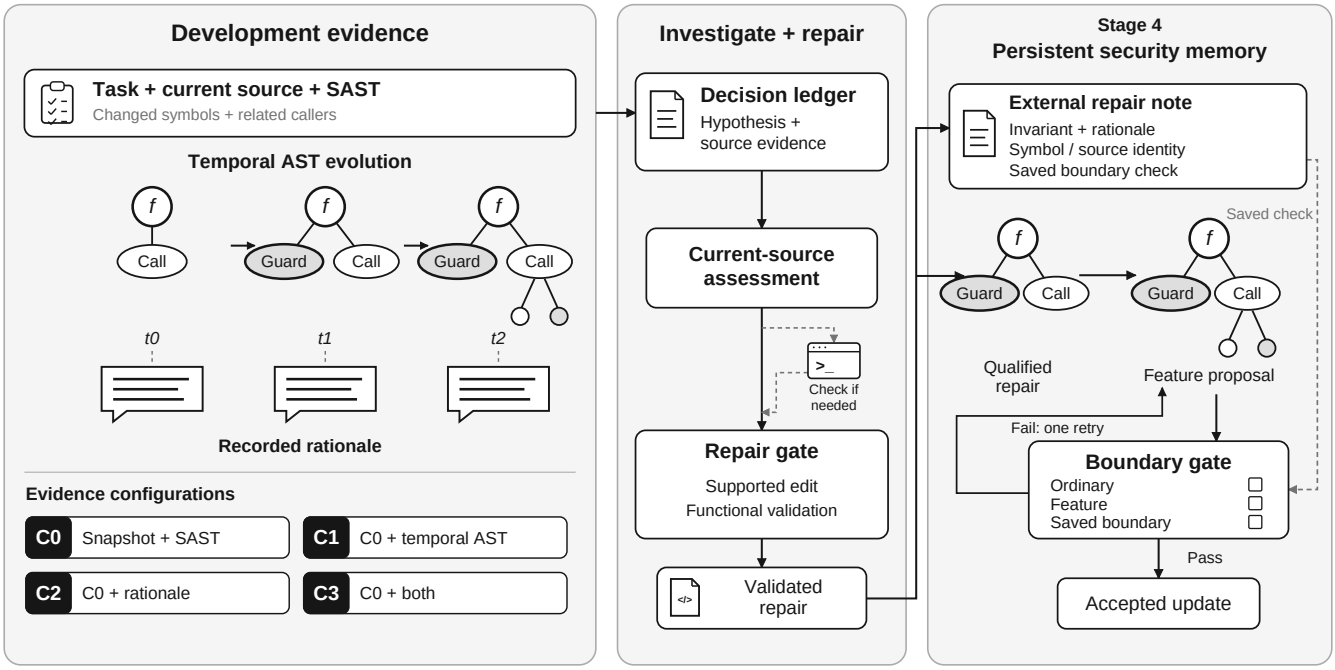


Fig. 1. TRACER links symbol-level development evidence to guarded security repair, then preserves repair invariants and executable checks for subsequent feature evolution. C0–C3 vary the evidence available during investigation.

rity investigation, repair, and maintenance workflow packaged as a lightweight, portable skill for integration with coding agents and continuous-integration pipelines. Given a bounded Git range or a compatible agent trajectory, TRACER performs change-informed symbol planning, keeps typed AST evolution separate from recorded rationale, grounds repair decisions in current-source assessment and conditional runtime checks, and emits persistent repair notes that can be coupled with executable boundary gates. Across 3,096 SusVibes repair runs spanning seven coding generators and two security investigators, its planner includes reference-vulnerable symbols in 94.6% of attempts, compared with 13.4% primary SAST symbol overlap. The rationale-enabled configuration achieves the highest verified security pass counts with both investigators. Combining both history channels does not improve on it. A complementary study of structural-biology software [24]–[29] verifies five component repairs and tests their subsequent feature evolution. Persistent memory with a one-retry boundary gate achieves joint feature-and-boundary success in 5/5 cases, compared with 3/5 for unguarded continuation.

The paper makes three contributions.

- **A provenance-aware architecture and portable skill.** TRACER connects change-informed planning, separate AST-evolution and rationale channels, guarded static/dynamic investigation, and persistent repair memory in a workflow that can accompany an agent-generated change through later maintenance.
- **A multi-investigator evaluation.** The SusVibes study compares four context bundles across seven coding generators and two investigators, measuring review coverage, verified security outcomes, and functional preservation. The structural-biology study evaluates the maintenance stage under silent-fix feature evolution.
- **A stage-level funnel and failure taxonomy.** A seven-version engineering progression and a full-cohort investigation funnel identify losses during diagnosis, validation scheduling, patch retention, and semantic repair, providing concrete design lessons for agentic review systems.

II. BACKGROUND AND MOTIVATION

A. Industry Relevance and Code Archaeology

When an agent-generated pull request works, an industrial reviewer must still determine which implementation assumptions deserve scrutiny and which security conditions must survive later maintenance. CodeQL and Semgrep can report locations and flows in the final snapshot [8], [9], but the reviewer must connect those findings to the submitted change, affected callers, and the intended contract. We call this reconstruction *code archaeology*. It recovers the context behind the current implementation from the evidence left available. Industrial static-analysis studies emphasize actionable diagnostics, developer trust, and integration into existing review workflows [10]–[12]. Preserving the development trace can supply context for that review, especially when the final diff omits an intermediate decision or correction.

For scientific-software teams such as Nostrum Biodiscovery, checkpoint loading, structure parsing, archive extraction, and output-path handling are concrete interfaces where compat-

ibility requests can invalidate earlier assumptions. A useful workflow must investigate these boundaries while preserving the requested scientific functionality, then retain the repair's purpose for the next change. Connecting review evidence to reusable checks fits secure-development practices for code review, testing, and retaining vulnerability knowledge [30]. Our domain study examines these component-level obligations. Production effectiveness remains to be evaluated.

B. Tracking Structural Evolution Within a Bounded Context

Making history available is not equivalent to making it usable. Source lines shift, definitions are added or renamed, and refactoring changes how code is divided among functions and classes. Intermediate fragments may not parse. A temporal representation therefore cannot assume that node names or line positions remain stable across versions. A line-based `git diff` records textual changes, but does not itself establish which function-level contract changed. Retaining every full AST and cross-version relation also expands the context with repeated or irrelevant structure. Fine-grained AST matching through GumTree [19], [31], together with function-change representations such as CoLeFunDa's [22], motivates a focused representation that maps observed changes to symbols, summarizes their structural edits, and preserves explicit gaps where continuity cannot be established.

C. Task Intent and Recorded Reasoning

Static analysis and AST differencing answer complementary questions about reported flows and structural changes, but neither alone supplies the requested behavior or every domain-specific assumption. Code can execute successfully while mishandling an input representation, trusting a caller without justification, or implementing only part of the task's contract. ReAct-style trajectories interleave explanations with actions [18], allowing the task description, stated assumptions, and observed edits to be examined together. These recorded explanations are auditable claims, not guaranteed accounts of internal chain-of-thought [20], [21]. A statement that an input is already validated, for example, creates a concrete obligation to inspect the relevant validator and its callers.

Our motivating cases come from repository tasks in SusVibes [6], [7]. In Python-RSA,[1] comparing `decrypt` and `verify` exposes an asymmetric encoded-input-length guard. LangChain[2] similarly illustrates an incomplete origin condition that checks network location but omits scheme equality. Wagtail CMS[3] shows the danger of repairing an intermediate label without inspecting downstream rendering. An early candidate introduces double-escaping. In Django,[4] recorded rationale favors character-reference conversion in `strip_tags`, while its tested contract requires preserving relevant text representations. A partial handler repair still fails. These cases motivate both discovering missing invariants and checking a proposed repair at the correct boundary.

[1]https://github.com/sybrenstuvel/python-rsa
[2]https://github.com/langchain-ai/langchain
[3]https://github.com/wagtail/wagtail
[4]https://github.com/django/django

III. TRACER Architecture and Portable Skill

A. Symbol-Level Tracking and Snapshot SAST (C0)

The evaluated Python pipeline builds a symbol-level index using the standard-library `ast.parse` parser [32] and a custom AST visitor. For each parsed snapshot, the index records functions, asynchronous functions, methods, and classes by repository path, qualified name, and definition span. The history adapter also records structural hashes. Rather than using diff lines as the review unit, the planner maps submitted patch hunks and deletion anchors to their enclosing current symbols, excludes test and documentation paths, and records module-level or unavailable mappings when needed. It links guards, candidate in-file callers, and related changed symbols, while attributing Semgrep and CodeQL locations and flow evidence to their originating scanners [8], [9]. Local call-name matches are syntactic review hints, not resolved dataflow. This channel defines C0 as a change-informed snapshot review that scopes investigation beyond scanner locations, but still requires archaeology to explain how and why the current symbol was written.

B. Temporal AST Evolution via GumTree (C1)

C1 adds the observed structural evolution of selected symbols. TRACER parses available before/after source pairs into typed Python ASTs and computes edit scripts with GumTree 3.0.0 [19], [31]. It filters operations to the target symbols, retains guards and calls inside parent-subtree additions, normalizes recoverable fragment indentation, and bounds operations, symbol summaries, and delivered characters. The resulting hints describe changes without passing every historical snapshot to the investigator. Cutoff-bounded Git ancestry and recorded agent edits retain separate provenance. Path/name associations and unique fragment anchors link evidence to current symbols. Continuity gaps, parse failures, and truncation remain explicit rather than implying complete identity tracking through arbitrary renames or refactors.

C. Symbol-Linked Reasoning Traces (C2 and C3)

The task description is shared context for every configuration. C2 additionally retrieves original ReAct-style development prose [18], excluding fenced code so explanations remain separate from source changes. The extractor links excerpts through recorded action paths, symbol mentions, and uniquely matched edit fragments, retaining event IDs, offsets, and source hashes. A file association remains distinguishable from a current-symbol anchor. When an excerpt informs a hypothesis, its quotation, testable assumption, and verification plan are stored with that hypothesis's target symbol in the decision ledger. The investigator records whether current evidence supports, contradicts, or leaves the assumption unresolved, since rationale is not authoritative evidence of behavior [20], [21]. C3 combines these rationale links with C1's temporal AST evidence and divergence cues. Rationale retrieval remains available even when an old AST cannot be reconstructed.

D. Decision Ledger, Repair Gate, and Portable Skill

The investigator records a hypothesis's target symbol, input boundary, and alleged violated property in a structured decision ledger, serialized in `edit_note.json`. It grounds assessments in quotations from the current source and, when evidence remains inconclusive, bounded Python checks following PAGENT's analysis-guided testing pattern [17]. Checks record the source version, target reachability, ordinary-input behavior, and boundary observations. The repair gate requires a supported defect, review of relevant downstream consumers, and validation of the candidate against public functional tests. Assessments and checks must apply to the current candidate. The note preserves their evidence links alongside patch, source, and trajectory identities.

For Stage 4 maintenance, a qualified repair note additionally binds the security invariant to the exact repaired-symbol hash and an independently owned executable boundary check. Subsequent feature proposals must satisfy ordinary, feature, and saved boundary checks. The continuation gate returns failure feedback for a bounded retry or retains the qualified starting code, recording acceptance and fallback separately.

The portable `tracer-security` skill[5] packages this symbol-tracking and ledger workflow for a host coding agent. Local helpers consume a bounded Git history or compatible recorded trajectory and produce review evidence using the available scanner and history adapters. The host supplies its editing and test interface, while a CI pipeline can run the saved checks on later changes. This separation supports adoption without depending on the SusVibes runner or its hidden evaluation tests.

IV. Experimental Design and Reproducibility

A. Research Questions

We structure the evaluation around five research questions that follow an agent-generated change from initial localization to subsequent maintenance. These are retrospective reporting questions for the completed experiments.

RQ1 (Localization Coverage). *How effectively does change-informed planning expose vulnerable symbols compared with stock SAST locations?*

Diagnosis requires bringing the relevant source into scope. We measure whether patch-selected symbols and their relations expose reference-relevant code beyond the locations reported by the scanners.

RQ2 (End-to-End Verified Repair). *Does guarded investigation improve verified security outcomes without degrading functionality?*

Security interventions can break working features. We measure security conversions and functional regressions after current-source assessment, conditional runtime checks, and public functional validation.

RQ3 (Provenance Channels). *How do temporal AST history, recorded rationale, and their combination affect verified repair?*

[5]https://github.com/pkrajput/TRACER_skill

TABLE I
Cohort inventory, locally verified original outcomes, and review coverage. Verif. counts terminal-complete original evaluations, with Func/Sec reported on that subset. Coverage uses all 387 admissions.

Generator	Source	Admit.	Verif.	Func	Sec
Claude 4 Sonnet	186	88	83	81	15
Gemini 2.5 Pro	186	29	24	24	6
Gemini 3 Pro	186	58	55	54	16
GLM-4.7-Flash	186	38	30	30	5
GLM-5	186	100	98	98	17
Kimi K2	186	33	28	27	4
Qwen3-Coder-Next	186	41	36	36	6
Total	1302	387	354	350	69
Coverage among all 387 admitted attempts					
SAST reference file				228	58.9%
SAST primary reference symbol				52	13.4%
SAST symbol including flow endpoints				63	16.3%
Change-informed plan reference symbol				366	94.6%

More context does not automatically produce better repairs. Comparing the four implemented bundles on identical attempts measures their relative contribution, with accompanying audit cues treated as part of each bundle.

RQ4 (Stage-Level Bottlenecks). *Where are candidates lost between symbol localization and verified repair?*

Review coverage does not guarantee a successful patch. A seven-version development study and a four-stage investigation funnel distinguish diagnostic, validation, semantic, and operational losses.

RQ5 (Persistent Memory Under Silent-Fix Evolution). *Can repair memory and boundary gates preserve repaired properties while subsequent feature changes succeed?*

Later maintenance must satisfy both the new feature request and the earlier repair contract. We test whether saved repair notes and executable checks support that joint outcome in structural-biology software under silent-fix conditions.

B. Benchmark, Cohort Inventory, and Evaluation Predicates

We use SusVibes and its released functional/security evaluator [6], [7]. An *attempt* pairs a task with an archived coding-generator output, while a *cell* additionally identifies the investigator and configuration. Among 1,302 archived outputs from seven generators, 472 have publisher-reported functional success, and local environment reproduction and source-integrity screening admit 387 attempts across 118 tasks (Table I). Each investigator contributes 1,548 cells, giving 3,096 recorded repair runs.

FuncPass and SecPass require recognized framework-specific terminal summaries, including the benchmark's expected-failure allowances. The shared comparison contains 345 attempts with completed functional and security evaluations for the original output and all eight investigator/configuration candidates. DeepSeek additionally supports a 354-attempt four-configuration comparison. The other 42 admissions remain outside the shared comparison. A frozen task-level exposure split identifies 235 development-exposed and

110 previously protected shared attempts, with labels consistent across generators. These are conditional repair outcomes, not new benchmark-wide generator pass@1 estimates.

Hidden security tests and reference patches are used only for retrospective evaluation. Investigators receive public functional feedback. Recovery evaluates originals and candidates under the same restored environment, records exact-hash reuse, and preserves earlier completed cells and their execution revisions. Conversions, regressions, and unresolved execution remain separate. Task-cluster intervals use 5,000 percentile-bootstrap replicates with seed 12 to account for repeated generator attempts [33].

C. Investigated Configurations and Scanner Setup

DeepSeek-V3.2 and Gemini-3.8-Flash investigate each attempt with temperature zero, a 22-turn cap, and a shared 100,000-byte message setting. DeepSeek uses OpenRouter's SiliconFlow FP8 endpoint, while Gemini uses its native endpoint with LOW thinking. Every configuration receives the functional request, patch-selected current source, scanner context, and the same guarded protocol.

- **C0. Snapshot + SAST.** Change-informed current-source review with scanner findings.
- **C1. + Temporal AST.** C0 with observed structural code changes.
- **C2. + Recorded Rationale.** C0 with original development explanations.
- **C3. Combined.** C0 with both channels and linked divergence evidence.

C0 already receives agent-patch-selected targets. C1–C3 additionally receive deterministic completeness cues, without a cue-only control. Comparisons therefore concern implemented context bundles. Upstream Git ancestry is withheld to avoid revealing masked implementations, so measured temporal evidence comes from post-reference agent trajectories.

All configurations share Semgrep CE 1.101.0 with Python rules at commit `495df89c` and CodeQL 2.20.0 with Python queries 1.3.4 and the security-extended suite. Semgrep includes correctness and maintainability rules alongside security checks. Scans use source exports without hidden evaluation artifacts. Planning scopes findings to changed files and at most twelve groups, while overlap audits use the full scanner output. SAST-seeded attempts receive at least one scanner alert in their input, while unseeded attempts receive none. Task-level outcomes do not establish alert-level false-positive reduction.

V. Empirical Results on Coverage and Repair Outcomes

A. RQ1. Evidence Available for Review

Stock SAST overlaps a reference file in 228/387 attempts (58.9%), a primary reference symbol in 52/387 (13.4%), and a symbol including flow positions in 63/387 (16.3%). The planner includes a reference symbol in 366/387 plans (94.6%) and its complete body in 329/387 (Table I). Among 291 publisher-SecFail attempts, primary and flow-inclusive overlap are 41 and 50, while plan inclusion reaches 281. C0 therefore receives relevant context even when SAST is silent. Of 335 attempts without a primary reference-symbol warning, patch selection supplies a reference group in 317 and its complete body in 282, while functional requests mention the target in 307. This measures review opportunity, with file-only mappings kept separate from named symbols.

> **RQ1 Finding.** Change-informed planning includes reference-relevant symbols in 94.6% of review plans, compared with 13.4% primary SAST symbol overlap. Diagnosis and repair remain subsequent steps.

B. RQ2. Verified End-to-End Repair

The shared cohort begins with 342 functional and 68 security passes among 345 attempts. DeepSeek C0–C3 reach 71/73/76/74 SecPass, while Gemini reaches 70/74/75/73 (Table II). C2 ranks highest with both investigators, raising SecPass from 19.71% to 22.03% and 21.74%, respectively, and adding five net passes over C0 in each campaign (1.45 percentage points). Functional passes remain 341/342/342/342 for DeepSeek and 341/340/341/340 for Gemini. The 354-attempt DeepSeek comparison preserves the exact ranking, with 72/74/77/75 SecPass from 69 original passes.

> **RQ2 Finding.** C2 improves baseline SecPass by 8/345 cases with DeepSeek and 7/345 with Gemini. Corresponding functional losses are zero and one.

C. RQ3. Context Bundles and Conversion Identity

DeepSeek C2 makes eight security conversions across six distinct tasks, while Gemini C2 makes seven across four. These are repairs to particular generated implementations, not fifteen independent discoveries. For C2 versus C0, task-cluster 95% intervals are [0.00, 2.96] percentage points for DeepSeek and $[-0.32, 3.86]$ for Gemini, with six wins and one loss in each paired comparison. C1 and C3 exchange positions between investigators, so adding both history channels does not consistently improve repair.

On the 110 previously protected attempts, 24 original security passes become 26/25/26/24 for DeepSeek and 25/27/27/25 for Gemini. C2 ties C0 for DeepSeek and exceeds it by two for Gemini. On the 235 development-exposed attempts, 44 original passes become 45/48/50/50 and 45/47/48/48, respectively. Table II retains the generator and scanner-seeding breakdowns alongside these exposure strata.

> **RQ3 Finding.** The rationale-enabled bundle ranks highest with both investigators. Its protected-stratum gain over C0 is zero cases for DeepSeek and two for Gemini. Combined history does not improve on it.

VI. Iterative Engineering and Failure Taxonomy

A. Iterative Engineering Progression (v6–v12)

To understand how protocol design affects repair losses (RQ4), we trace seven iterative development revisions of our

TABLE II

VERIFIED REPAIR OUTCOMES. F/S DENOTES FUNCPASS/SECPASS. SHARED PANELS USE THE SAME 345 ATTEMPTS WITH COMPLETE OUTCOMES FOR ALL EIGHT INVESTIGATOR/CONFIGURATION CANDIDATES. EACH PANEL STATES ITS POPULATION.

Population / investigator	*N*	Original	C0	C1	C2	C3
A. End-to-end outcomes, FuncPass / SecPass counts						
DeepSeek (four-arm matched)	354	350 / 69	349 / 72	350 / 74	**350 / 77**	350 / 75
DeepSeek (shared eight-arm)	345	342 / 68	341 / 71	342 / 73	**342 / 76**	342 / 74
Gemini (shared eight-arm)	345	342 / 68	341 / 70	340 / 74	**341 / 75**	340 / 73
B. Shared-cohort SecPass by generator and stratum. Each arm cell is DeepSeek / Gemini						
Claude 4 Sonnet	83	15	16 / 16	16 / 16	18 / 18	16 / 16
Gemini 2.5 Pro	24	6	6 / 6	6 / 6	6 / 6	6 / 6
Gemini 3 Pro	55	16	16 / 16	17 / 16	17 / 17	17 / 18
GLM-4.7-Flash	30	5	5 / 6	6 / 6	5 / 5	5 / 6
GLM-5	93	16	16 / 16	16 / 16	18 / 17	17 / 16
Kimi K2	26	4	6 / 4	6 / 8	6 / 6	6 / 5
Qwen3-Coder-Next	34	6	6 / 6	6 / 6	6 / 6	7 / 6
SAST-seeded	176	36	37 / 38	37 / 40	40 / 40	38 / 40
Unseeded	169	32	34 / 32	36 / 34	36 / 35	36 / 33
Development exposed	235	44	45 / 45	48 / 47	50 / 48	50 / 48
Previously protected	110	24	26 / 25	25 / 27	26 / 27	24 / 25

TABLE III

ENGINEERING PROGRESSION AND NESTED FUNNEL. PANEL A USES TEN FIXED CASES, WITH S DENOTING SECPASS. REJECT./CITE/LIMIT/ERROR GIVE PAIRED C0/C3 COUNTS OF REJECTED ACTIONS, CITATION REJECTIONS, TURN-LIMIT ENDINGS, AND REPAIR ERRORS. PANEL B USES 387 ATTEMPTS PER ARM. D/G DENOTES DEEPSEEK/GEMINI, WITH SEC/NEW COUNTING SECURITY PASSES/CONVERSIONS WITHIN S4.

A. Fixed ten-case development cohort

Version	C0 S	C3 S	Reject.	Cite	Limit	Error
v6	1	0	31/48	15/23	3/2	0/0
v7	1	1	64/73	5/20	3/2	0/0
v8	1	0	23/9	0/0	2/1	0/0
v9	1	1	92/62	0/0	9/9	0/1
v10	1	3	49/61	0/0	3/4	1/1
v11	1	3	57/30	0/0	4/4	0/2
v12	2	4	77/43	0/1	5/6	0/0

B. Full recorded funnel, 387 attempts per arm

Inv.	Arm	S1	S2	S3	S4	Sec	New
D	C0	245	109	89	58	13	4
D	C1	251	109	98	65	13	5
D	C2	237	85	74	54	13	8
D	C3	252	97	89	60	10	6
G	C0	213	15	14	11	3	2
G	C1	225	32	28	20	8	6
G	C2	223	29	27	18	8	7
G	C3	235	23	21	17	5	4

workflow (`v6` through `v12`, where `v12` is the final released configuration evaluated in Section V) across a fixed ten-case diagnostic cohort (Table III, Panel A). It contains nine initially insecure outputs and one secure Wagtail control. Both configurations retain ten functional passes throughout. Final C3 preserves the control and repairs three cases, compared with one repair for C0.

Evidence fidelity (v6–v8). Current-source citation binding, structured probe observations, and symmetric scanner-negative gates make assessments auditable. Preserving GumTree parent-subtree operations and normalizing fragments restores missing guard descriptions, but does not resolve Wagtail's downstream rendering contract.

Boundary authorization (v9–v10). Static invariant-gap and differential guard-bypass routes support diagnoses. Recognizing preloaded callers and intrinsic input contracts removes redundant review requirements. RSA and LangChain then convert in C3.

Context and lifecycle completeness (v11–v12). Deduplicating state and reserving context headroom prevents two previously observed packing aborts. Class-span reachability and lifecycle cues support the third C3 repair, in Wagtail Forms. All twenty final qualification cells avoid repair errors, although C3 turn-limit endings rise from two to six across the progression.

B. Stage-Level Investigation Funnel and Validation Scheduling

Table III, Panel B, follows nested stages. S1 records a hypothesis on a reference symbol, S2 a supported assessment there, S3 a reference-file edit within that supported group, and S4 a nonempty retained patch. DeepSeek C3 reaches 252/97/89/60 cells across S1–S4, with ten retained SecPass outcomes including six conversions. Gemini C3 reaches 235/23/21/17, with five passes and four conversions. These retained-subset counts differ from whole-campaign totals. The largest loss is between localization and support. Diagnostic trace review finds refutations, inconclusive assessments, overgeneralized hypotheses, and turn exhaustion alongside rejected positive actions. A reference location alone does not establish a defect, including on initially secure outputs.

A separate development recovery check evaluates seventy saved candidates discarded for exhausted functional-validation slots (37), reaching the turn limit before validation (31), or missing a post-edit probe rerun (two). Without regenerating patches, 39 pass public functionality, eighteen fail, and thirteen remain unresolved. Nine pass security, comprising eight preserved successes and one conversion. This motivates reserving final public-validation capacity and reusing validation only for

TABLE IV
SEMANTIC REVIEW OF FIFTEEN TASKS OUTSIDE THE DIAGNOSTIC COHORT, WITH RETAINED REFERENCE-SYMBOL EDITS AND VERIFIED FUNCPASS/SECFAIL. CATEGORIES DESCRIBE THIS PURPOSIVE SAMPLE.

Category	N	Representative project and remaining defect
Wrong property	4	Aiohttp session expiry ignored. FastAPI body size replaces media-type handling. Route anchoring and normalized usernames remain.
Right class, incomplete fix	9	JWCrypto checks after decompression. Streamlit symlink escape. Airflow Hive inherited schema bypass. Waitress malformed Content-Length. Flask-Admin backslash URL bypass.
Right condition, wrong failure behavior	2	NumPy raises IndexError instead of ValueError on an empty axis. Markdown-it-py malformed-byte decoding still raises.

an identical patch. Recovery records remain separate from original outcomes.

C. Semantic Repair Failures and Operational Errors

We compare retained diffs, reference patches, and official failing assertions on fifteen tasks outside the diagnostic cohort, fourteen from DeepSeek and one from Gemini, with functional success before and after editing (Table IV). Four patches address the wrong property, nine identify the right vulnerability class but leave an incomplete fix, and two violate the required failure behavior. Examples include lexical path normalization that misses symlink escape, validation that omits inherited connection fields, and a correct rejection using the wrong exception type. Symbol localization and plausible edits therefore need full input, lifecycle, and error-contract checks.

Operational errors remain separately observable. Gemini records 205 repair errors, comprising 153 `blockReason: OTHER` responses, 45 JSON decoding errors, six HTTP 500 errors, and one other value error. DeepSeek records 107, comprising 63 stop interruptions, 32 HTTP 429 responses, nine deadlines, two value errors, and one connection error. Provider labels are not assigned inferred causes. Bounded transport retries retain billing reservations, and completed responses are not regenerated. An errored cell can still have an officially evaluated retained candidate or no-op in the matched analysis.

RQ4 Finding. Losses arise during diagnosis, validation scheduling, semantic repair, and provider execution. Linking actions, patches, and terminal test outcomes distinguishes the engineering intervention each failure needs.

VII. STRUCTURAL BIOLOGY PIPELINES AND PERSISTENT SECURITY MEMORY

RQ5 evaluates Stage 4, persistent security memory and boundary gating, through feature evolution in public structural-biology software.

A. Change-Informed Discovery and Repair

The six repositories and component checks were selected with our Nostrum Biodiscovery collaborator. They cover RFdiffusion [24], ProteinMPNN [25], haddock3 [26], DockQ [27], AutoDock-Vina [28], and foldseek [29]. Semgrep 1.101.0 reports 420 warnings, none overlapping the five selected target symbols. Unlike the benchmark, this pilot uses Semgrep alone. Bounded source review and repository history identify component contracts. Each case has an ordinary control and an independently owned boundary assertion that fails before repair and passes afterward. Qualification uses a pinned Python 3.11 container, read-only source/check mounts, and disabled network access. Foldseek contributes eleven warnings but no suitable project-owned Python repair case.

Table V spans resource lifecycle, symlink containment, path traversal, deserialization, and branch-bypassed validation. Repairs close DockQ handles on parser failure, check resolved extraction paths in haddock3, restrict ProteinMPNN record names used as paths, and validate AutoDock-Vina numeric inputs independently of overwrite permission. RFdiffusion uses `weights_only=True` with scoped `safe_globals([slice])` under PyTorch 2.5.1 [34], addressing untrusted pickle loading [35] while preserving accepted checkpoint forms. Four repairs are model-written. RFdiffusion's user-specified remedy is implemented locally after two candidates fail ordinary validation. Every repair note preserves the invariant, rationale origin, source/patch identities, Git/GumTree evidence, and qualified check.

B. Preventing Overwrites Under Silent-Fix Feature Evolution

Following the silent-fix motivation [22], we remove comment tokens and AST-identified docstrings using Python's `tokenize` and `ast` interfaces [32], [36]. The executable AST is unchanged after excluding documentation, and cleaned starts pass ordinary and boundary checks again. Both conditions receive identical code and feature requests, with frozen feature checks (Table V). Memory+Gate additionally receives the external repair note. DeepSeek V3.2 at temperature zero makes one No Memory proposal. Memory+Gate permits one retry after exact application, ordinary, feature, or boundary-check feedback, then retains the secure start if needed. Fallback does not count as feature delivery.

Joint feature-and-boundary success is 3/5 for No Memory, 4/5 for Memory+Gate's first proposals, and 5/5 after retry. All first proposals pass ordinary checks. Boundary preservation is 4/5 in each condition, with RFdiffusion unknown because of an exception. The no-memory haddock3 candidate preserves containment but omits the expected flattened ZIP output. Both initial RFdiffusion candidates reference an unimported `pickle` name. Gate feedback enables its import and successful compatibility checks. The two additional joint successes therefore concern archive-format support and exception handling, rather than two observed vulnerability reintroductions. Figure 2 shows the external haddock3 invariant. All five guarded outputs implement the feature, with no fallback.

TABLE V

STRUCTURAL-BIOLOGY MAINTENANCE STUDY. JOINT SUCCESS REQUIRES ORDINARY BEHAVIOR, THE REQUESTED FEATURE, AND THE SAVED BOUNDARY ASSERTION. THESE FIVE REPOSITORIES CONTRIBUTE 409 OF 420 SEMGREP WARNINGS. FOLDSEEK CONTRIBUTES ELEVEN WARNINGS AND NO REPAIR CASE. NO WARNING OVERLAPS A SELECTED TARGET SYMBOL.

Repository	Alerts	Target symbol	Repaired component defect	Feature evolution	Joint success	
					No Memory	Memory+Gate
DockQ	10	`load_PDB`	Handle leak on parser failure/fallback	Extension-first CIF/PDB parsing, optional format return	Yes	Yes
haddock3	247	`extract_files_flat`	Extraction escape through destination symlink	ZIP and TAR support, optional nested paths	No[a]	Yes
ProteinMPNN	43	`StructureDataset .__init__`	Unvalidated record names used in output paths	Minimum-length and missing-coordinate filters	Yes	Yes
RFdiffusion	49	`Sampler. load_checkpoint`	Unrestricted pickle loading under pinned runtime[b]	Legacy-checkpoint exception handling and CPU fallback	No[c]	Yes, retry
AutoDock-Vina	60	`Vina.write_poses`	Overwrite branch bypasses numeric validation	Verbose flag and consolidated validation chain	Yes	Yes
Joint successes, first proposal					**3/5**	**4/5**
Joint successes, final accepted feature implementation					**3/5**	**5/5**
Additional proposals / secure-code fallbacks					0 / 0	1 / 0

[a] ZIP output is missing. The saved boundary check passes. [b] User-specified remedy, locally implemented after two unsuccessful model repairs. [c] An unimported `pickle` name leaves the feature and boundary outcomes unknown. The gate's retry resolves this in Memory+Gate.

Silent repair, resolved-path containment

```
target_path = dest / member.name
target_resolved = target_path.resolve()
if dest not in target_resolved.parents \
  and dest != target_resolved:
    continue
```

External repair-note fields (excerpt)

`symbol`: `extract_files_flat`
`security_invariant`: "their final extraction paths must be within the destination directory after resolving any symlinks in the destination path."
`fix_verified`: `true`
`fix_patch_sha256`: `e6973008...`
`check_qualification`: original assertion fails. Fixed version and ordinary controls pass.

Feature evolution, ZIP/TAR + nested paths

No Memory. Ordinary/boundary pass. ZIP output missing.
Memory+Gate. Ordinary, feature, and boundary all pass.

Fig. 2. Saved haddock3 repair memory. The containment guard persists from the silent start to ZIP/TAR evolution. Ordinary extraction, the new feature, and the saved boundary are checked separately.

RQ5 Finding. Memory with a one-retry boundary gate achieves 5/5 joint feature-and-boundary successes, compared with 3/5 for unguarded evolution, without fallbacks. This compares complete workflows, not memory alone.

VIII. LESSONS FOR SE TEAMS AND THREATS TO VALIDITY

A. Practical Lessons

Context dilution. C2 outperforms C3 with both investigators. A relevant explanation may connect an assumption to a current-code obligation more directly than combined structural and textual history. Context dilution is a plausible explanation, not an isolated causal finding, since context length and attention are uncontrolled. Teams should retrieve history to resolve a specific hypothesis rather than maximize context volume.

Verifiable boundary obligations. SAST supplies reproducible entry points, while patch-selected symbols expose changes without warnings. A missing sibling guard or recorded trust assumption becomes actionable when translated into a check of current inputs, consumers, or object lifecycle. Preserve the link from the historical observation through the hypothesis to its validating evidence.

Validation and maintenance gates. Reserve final validation capacity, retain rejected patches and reasons, and reuse results only for identical candidates. Recognized terminal summaries distinguish completed checks from interrupted tests. Persistent memory carries the accepted invariant and independently owned check into maintenance, where the gate returns concrete feedback or preserves the secure start. Report feature delivery and fallback preservation separately.

B. Threats to Validity and Scope

Construct and internal validity. Reference-patch overlap measures review opportunity, with broad classes and incidental edits potentially overstating semantic localization. Task tests cover specified contracts, not whole-repository security or unrelated alerts. Test adequacy and overfitting remain limits [37], [38], and recorded rationale need not faithfully describe internal reasoning [20], [21]. Adaptive development changes several mechanisms together. C0 receives patch-selected targets, C1–C3 add audit cues without a cue-only control, and temporal evidence excludes upstream ancestry. Resumed execution retains earlier revisions, with paired original/candidate evaluations after environment recovery. The previously protected stratum is separately reported but is not an untouched prospective replication.

External and domain-study validity. Selection favors Python outputs with publisher-reported functional success. Missing complete outcomes for 42 admissions may be nonrandom,

and shared tasks create dependence addressed by task-cluster intervals. Few conversions limit precision. The fifteen semantic reviews are purposive and single-reviewer. Five component repairs in six structural-biology repositories are researcher-run with industry input, not independently confirmed CVEs or a practitioner deployment. Memory+Gate includes a retry absent from No Memory, so its gain cannot be attributed to memory alone. Source/check hashes establish traceability rather than semantic correctness. Broader repository coverage and independent practitioner validation remain future work.

IX. Related Work

SWE-bench measures issue resolution [39], whereas SusVibes separates functional and security outcomes [6], [7]. CVEfixes links vulnerabilities to fixing histories, and Vul4J provides reproducible vulnerabilities and triggering tests [40], [41]. SWE-agent studies repository interaction interfaces [1], while ReAct supplies interleaved explanation/action traces [18]. For security investigation, PAGENT combines static guidance with dynamic profiling [17], IRIS infers taint specifications and filters candidate paths [13], LLMDFA couples model reasoning with parsing and feasibility checks [14], and RepoAudit explores data flows using memory and validation [15]. Our comparison asks what development evidence adds to a common current-source repair protocol.

Historical analysis operates on complementary units. ViolationTracker reconstructs static-warning histories [42], CoLeFunDa represents function changes to identify silent fixes [22], KNighter synthesizes checkers from historical patches [43], and FixMiner extracts reusable repair patterns [44]. GumTree provides structural edit matching [19], [31]. Analyzer-guided repair includes InferFix [16], while ChatRepair uses conversational test feedback [45] and RepairAgent autonomously inspects, edits, and tests code [2]. These mechanisms motivate tracking losses between available evidence, supported diagnosis, and retained repair.

For subsequent maintenance, Reflexion retains verbal feedback in episodic memory [46], Self-Refine iterates through model-generated feedback [47], and POSTURA describes a persistent security graph [48]. The exploratory *Undoing the Fix* report studies deliberately elicited regressions under an AI-control setting [23]. Our benign feature-evolution study instead connects an earlier repair invariant to independently owned executable checks, distinguishing first proposals, feedback-assisted acceptance, and secure fallback.

X. Conclusion

TRACER links change-informed planning, temporal AST evolution, recorded rationale, and guarded repair to persistent maintenance evidence. Across 3,096 cells, C2 achieves the highest shared-cohort SecPass counts with both investigators, reaching 76/345 with DeepSeek and 75/345 with Gemini from 68 original passes. Stage-level losses explain why broad review coverage produces modest repair gains. In structural-biology components, memory with a one-retry gate completes five feature evolutions while preserving saved boundaries, compared with three under unguarded evolution. The portable skill and archived evidence support auditable security review and maintenance.

Data Availability

The skill, analysis helpers, derived per-cell outcomes, and tables are archived on Zenodo [49] and maintained at https://github.com/pkrajput/TRACER_skill. SusVibes provides the benchmark inputs [7]. The archive supports outcome recomputation, but excludes raw investigator transcripts, complete evaluator logs, and container exports.

Acknowledgements

We thank Google for Gemini API credits and Nostrum Biodiscovery for verification assistance and feedback on the work's practical relevance.

References

[1] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[2] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 2188–2200.

[3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 754–768.

[4] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do Users Write More Insecure Code with AI Assistants?" in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 2785–2799.

[5] Y. Fu *et al.*, "Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 8, pp. 218:1–218:34, 2025.

[6] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, "Is vibe coding safe? benchmarking vulnerability of agent-generated code in real-world tasks," in *Forty-third International Conference on Machine Learning*, 2026, author-provided conference citation; benchmark release v1.0.

[7] LeiLiLab, "SusVibes: Benchmark and evaluation pipeline," Software repository, 2026, evaluator revision used in this study; accessed October 19, 2026. [Online]. Available: https://github.com/LeiLiLab/susvibes/tree/7e1b4b05240e56dc2b4e8253b2cc5c9481d016f3

[8] GitHub, "CodeQL CLI," GitHub documentation, accessed October 19, 2026. [Online]. Available: https://docs.github.com/en/code-security/concepts/code-scanning/codeql/codeql-cli

[9] Semgrep, "Semgrep code overview," Semgrep documentation, accessed October 19, 2026; product capabilities vary by engine and configuration. [Online]. Available: https://docs.semgrep.dev/semgrep-code/overview

[10] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 672–681.

[11] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, "Lessons from building static analysis tools at Google," *Communications of the ACM*, vol. 61, no. 4, pp. 58–66, 2018.

[12] C. Calcagno *et al.*, "Moving Fast with Software Verification," in *NASA Formal Methods*, 2015, pp. 3–11.

[13] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in *International Conference on Learning Representations (ICLR)*, 2025.

[14] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, "LLMDFA: Analyzing Dataflow in Code with Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 131 545–131 574.

[15] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 21 083–21 100.

[16] M. Jin *et al.*, "InferFix: End-to-End Program Repair with LLMs," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1646–1656.

[17] A. Desai, M. Shafiuzzaman, W. Guo, and T. Bultan, "Program analysis guided LLM agent for proof-of-concept generation," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2604.07624

[18] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[19] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, "Fine-grained and accurate source code differencing," in *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2014, pp. 313–324.

[20] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[21] T. Lanham *et al.*, "Measuring Faithfulness in Chain-of-Thought Reasoning," 2023, preprint. [Online]. Available: https://arxiv.org/abs/2307.13702

[22] J. Zhou *et al.*, "CoLeFunDa: Explainable silent vulnerability fix identification," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2565–2577.

[23] Ivan, "Undoing the fix: Evaluating AI control against security regression attacks," Apart Research AI Control Hackathon report, Mar. 2026, author credited as Ivan in the supplied report; exploratory study. [Online]. Available: https://github.com/strivn/undoing-the-fix/tree/6511beccd75b63c93628ec8dc062ebb07a0bcc5d

[24] J. L. Watson *et al.*, "De novo design of protein structure and function with RFdiffusion," *Nature*, vol. 620, pp. 1089–1100, 2023.

[25] J. Dauparas *et al.*, "Robust deep learning–based protein sequence design using ProteinMPNN," *Science*, vol. 378, no. 6615, pp. 49–56, 2022.

[26] M. Giulini *et al.*, "HADDOCK3: A Modular and Versatile Platform for Integrative Modeling of Biomolecular Complexes," *Journal of Chemical Information and Modeling*, vol. 65, no. 13, pp. 7315–7324, 2025.

[27] C. Mirabello and B. Wallner, "DockQ v2: improved automatic quality measure for protein multimers, nucleic acids, and small molecules," *Bioinformatics*, vol. 40, no. 10, p. btae586, 2024.

[28] J. Eberhardt, D. Santos-Martins, A. F. Tillack, and S. Forli, "AutoDock Vina 1.2.0: New Docking Methods, Expanded Force Field, and Python Bindings," *Journal of Chemical Information and Modeling*, vol. 61, no. 8, pp. 3891–3898, 2021.

[29] M. van Kempen *et al.*, "Fast and accurate protein structure search with Foldseek," *Nature Biotechnology*, vol. 42, pp. 243–246, 2024.

[30] M. Souppaya, K. Scarfone, and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities," National Institute of Standards and Technology, Tech. Rep. SP 800-218, 2022.

[31] GumTreeDiff, "GumTree, release 3.0.0," Software release, accessed October 19, 2026. [Online]. Available: https://github.com/GumTreeDiff/gumtree/releases/tag/v3.0.0

[32] Python Software Foundation, "ast—Abstract Syntax Trees," Python 3.11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.python.org/3.11/library/ast.html

[33] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

[34] PyTorch Contributors, "Serialization API and Safe-Globals Context Manager," PyTorch v2.5.1 source and API documentation, 2024, pinned version used by the component study; accessed October 20, 2026. [Online]. Available: https://github.com/pytorch/pytorch/blob/v2.5.1/torch/serialization.py

[35] Python Software Foundation, "pickle—Python Object Serialization," Python 3.11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.python.org/3.11/library/pickle.html

[36] ——, "tokenize—Tokenizer for Python Source," Python 3.11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.python.org/3.11/library/tokenize.html

[37] M. Monperrus, "Automatic Software Repair: A Bibliography," *ACM Computing Surveys*, vol. 51, no. 1, pp. 17:1–17:24, 2018.

[38] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 532–543.

[39] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *International Conference on Learning Representations (ICLR)*, 2024.

[40] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021, pp. 30–39.

[41] Q.-C. Bui, R. Scandariato, and N. E. D. Ferreyra, "Vul4J: A Dataset of Reproducible Java Vulnerabilities Geared Towards the Study of Program Repair Techniques," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 464–468.

[42] P. Yu *et al.*, "ViolationTracker: Building precise histories for static analysis violations," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023.

[43] C. Yang, Z. Zhao, Z. Xie, H. Li, and L. Zhang, "KNighter: Transforming static analysis with LLM-synthesized checkers," in *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025.

[44] A. Koyuncu *et al.*, "FixMiner: Mining relevant fix patterns for automated program repair," *Empirical Software Engineering*, vol. 25, no. 3, pp. 1980–2024, 2020.

[45] C. S. Xia and L. Zhang, "Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 819–831.

[46] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[47] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[48] motornomad, "POSTURA: Agentic attack surface posture intelligence," Software repository, repository owner handle; accessed October 19, 2026. [Online]. Available: https://github.com/motornomad/postura

[49] P. Rajput and S. Khan, "Securing Agent-Generated Code Through Development History," Zenodo, 2026, software and derived study evidence. [Online]. Available: https://doi.org/10.5281/zenodo.23530070